

Продвинутые манипуляции с объектами: Деструктуризация и Spread

Практическое проектирование нормализованных хранилищ, обработка многоуровневых мутаций без сторонних библиотек, реализация Undo/Redo и построение фабрик форм.

```
> zsh - node advanced-state-test.js

# Тестирование строгой ссылочной изоляции стейта
→ node advanced-state-test.js
✓ Deep Immutable State Transition: PASSED (New Root Ref, Isolated Sub-Tree)
✓ Normalized Dictionary Merge: PASSED (O(1) Access Matrix)
✓ Dynamic Schema Form Extraction: PASSED (Rest Props Filtered)
```

01

Задача 1: Глубокое 3-уровневое обновление стейта без мутаций

Архитектура Redux-подобных редьюсеров со сложной вложенностью

В реальных приложениях состояние часто представляет собой глубокое дерево. При изменении узла на 3-м уровне необходимо создать новые ссылки для всех его родителей по цепочке до корня, сохранив при этом неизменными ссылки на соседние ветки.

</> 1.1. Реализация изолирующего редьюсера

```
const appState = { user: { id:
  "u100", profile: { firstName:
  "Иван", settings: { theme:
  "light", notifications: { email:
  true, push:
  false } } } }, posts: ["post_1", "post_2"]
  // Соседняя ветка
};
```

```
// Задача: Обновить settings.notifications.push в true и theme в
"dark"
// с сохранением ссылочной целостности для posts и других
веток:
function
updateNotificationSettings(state,
newTheme, pushStatus) { return {
...state, user: { ...state.user, profile: { ...state.user.profile,
settings: { ...state.user.profile.settings, theme: newTheme,
notifications: { ...state.user.profile.settings.notifications, push:
pushStatus } } } } }; }

const nextState =
updateNotificationSettings(appState,
"dark",
true);

// ПРОВЕРКА ССЫЛОК (Критично для React.мемо и селекторов
Redux):
console.log("1. Корень изменился:", appState !== nextState);
// true console.log("2. Ветка user изменилась:",
appState.user !== nextState.user);
// true console.log("3. Ветка posts СОХРАНИЛА ссылку:",
appState.posts === nextState.posts);
// true (Ререндера постов не будет!)
```

⚙️ Почему сохранение ссылки posts критично?

Если бы мы сделали `structuredClone(state)` или `JSON.parse(JSON.stringify(state))`, мы бы разорвали ссылки на **все** объекты, включая список постов. Компоненты постов, обернутые в `React.memo`, начали бы бесполезно перерисовываться, вызвав деградацию производительности (FPS Drop).

02

Задача 2: Нормализация реляционных данных (byId & allIds)

Преобразование массива с сервера в словарь для доступа $O(1)$ и атомарных апдейтов

В масштабных приложениях массивы сущностей нормализуют в словарь `{ byId: {}, allIds: [] }`. Это позволяет обновлять конкретный элемент без обхода всего массива через `.map()`.

📁 2.1. Нормализатор и точечный патчер

```
// Сырые данные с REST API:
const rawServerUsers = [ { id:
  "u1", name:
  "Анна", status:
  "active" }, { id:
  "u2", name:
  "Борис", status:
  "pending" }, { id:
  "u3", name:
  "Вера", status:
  "offline" } ];

// 1. Превращение массива в нормализованную структуру с помощью
  Object.fromEntries:
const normalizedUsers = { byId:
  Object.fromEntries(rawServerUsers.map(u => [u.id, u])), allIds: rawServerUsers.map(u => u.id) };

// 2. Мгновенное обновление статуса юзера "u2" за O(1) через
  Spread вычисляемого свойства:
function
patchUserStatus(state, userId, nextStatus)
```

```
{ if (!state.byId[userId])
return state;

return { ...state, byId: {
...state.byId, [userId]: { ...state.byId[userId], status: nextStatus
} } }; }

const updatedNormalized =
patchUserStatus(normalizedUsers,
"u2",
"active"); console.log("Юзер u2:",
updatedNormalized.byId["u2"]);
// { id: 'u2', name: 'Борис', status: 'active' }
console.log("Юзер u1 остался неизменен:", updatedNormalized.byId["u1"] ===
normalizedUsers.byId["u1"]);
// true
```

Задача 3: Исключение полей (Omit Keys) и фильтрация DOM-пропсов

Создание функции высшего порядка для очистки пропсов от кастомных флагов

Частая проблема React: если передать в нативный тег кастомный проп (например, `<button isSpecial={true}>`), React выдаст предупреждение *React does not recognize the `isSpecial` prop on a DOM element*. Напишем утилиту очистки с помощью деструктуризации и Rest.

3.1. Реализация функции omitProps

```
omitUtility.js

// Универсальная функция исключения списка ключей из
// объекта:
function
omit(obj, keysToOmit) {
  return Object.entries(obj).reduce((acc, [key, value]) => {
    if (!keysToOmit.includes(key)) { acc[key] = value; }
    return acc; }, {}); }
```

```
// Тестовые пропсы, пришедшие в UI компонент:
const rawButtonProps = { label:
  "Отправить", onClick: () => {},
  variant: "primary",
  // Кастомный проп (не для DOM)
  isElevated: true,
  // Кастомный проп (не для DOM)
  disabled: false,
  "aria-label":
    "Submit form"
};

// 1. Быстрое извлечение конкретных полей через Rest в
  компоненте:
const { variant, isElevated, label,
  ...safeDomProps } = rawButtonProps;

// 2. Или динамический omit по списку:
const cleanDomAttributes =
  omit(rawButtonProps, ["variant", "isElevated",
    "label"]); console.log("Безопасные DOM атрибуты:",
  safeDomProps);
// { onClick: [Function], disabled: false, 'aria-label': 'Submit
  form' }
```

04

Задача 4: Реализация паттерна Undo / Redo через иммутабельные снимки

Построение стека истории состояний на базе Spread-операций

Иммутабельность объектов позволяет тривиально реализовать историю изменений (Undo / Redo), просто сохраняя ссылки на предыдущие снимки стейта в массивы `past` и `future`.

↻ 4.1. Движок машины состояний с поддержкой отмены

```
function
createHistoryContainer(initialState) {
  let history = { past: [], present:
initialState, future: [] };

  return {
    getState: () ⇒ history.present,
```

```
// Переход к новому состоянию:
set: (newPresent) => { history = {
past: [...history.past, history.present], present: newPresent,
future: []
// Сбрасываем future при новом действии
}; },

// Шаг назад (Undo):
undo: () => {
if (history.past.length ===
0) return;
const previous =
history.past[history.past.length - 1];
const newPast = history.past.slice(0, history.past.length -
1); history = { past: newPast, present:
previous, future: [history.present, ...history.future] }; },

// Шаг вперед (Redo):
redo: () => {
if (history.future.length ===
0) return;
const next = history.future[0]; const newFuture =
history.future.slice(1); history = { past: [...history.past, history.present], present:
next, future: newFuture }; } };
```

```
// Тестирование:  
const doc =  
createHistoryContainer({ text:  
  "Версия 1", fontSize:  
14 }); doc.set({ text: "Версия 2", fontSize:  
16 }); doc.set({ text: "Версия 3", fontSize:  
18 }); doc.undo(); console.log("После Undo:", doc.getState().text);  
// "Версия 2"  
  
doc.redo(); console.log("После Redo:", doc.getState().text); // "Версия 3"
```

05

Задача 5: Глубокий Merge конфигураций с валидацией типов

Создание безопасного сборщика настроек с приоритетами

Обычный `{ ...defaults, ...userConfig }` перезаписывает вложенные объекты целиком, затирая дефолтные параметры. Напишем рекурсивный `deepMerge`, аккуратно объединяющий деревья произвольной глубины.

5.1. Рекурсивный deepMerge

```
const isObject = (item) => item &&
typeof item ===
"object" && !Array.isArray(item);

function
deepMerge(target, source) {
  const output = { ...target };
```

```
function
deepMerge(target, source) {
const output = { ...target };

if (isObject(target) &&
isObject(source)) { Object.keys(source).forEach(key => {
if (isObject(source[key])) {
if (!(key
in target)) { output[key] =
source[key]; } else { output[key] =
deepMerge(target[key], source[key]); } }
else { output[key] = source[key]; }
}); }

return output; }

// Тест слияния настроек приложения:
const defaultEnv = { api: { host:
"api.site.com", port:
443, ssl:
true }, ui: { animations:
true, theme:
"system" } };
```

```
const customEnv = { api: { port:
8080, ssl:
false }
// host не указан, но должен сохраниться!
};

const mergedEnv =
deepMerge(defaultEnv, customEnv);
console.log(mergedEnv.api);
// { host: 'api.site.com', port: 8080, ssl: false } – host HE
пропал!
```